Event Ticket Booking API — Repo Build Guide

Oct 11, 2026 · @adawy

Overview

Build a REST API where organizers publish events with limited seats and attendees book tickets safely, in about 3 weeks. The project proves you can handle concurrency, payments, idempotency and background jobs in Laravel.

Repo name: `event-ticketing-api`

Layer	Choice
Language / framework	PHP 8.3, Laravel 12
Database	MySQL 8 (or PostgreSQL 16)
Cache / queue	Redis + Laravel Horizon
Auth	Laravel Sanctum (token-based)
Roles	spatie/laravel-permission
PDF / QR	barryvdh/laravel-dompdf, simplesoftwareio/simple-qrcode
Payments	Stripe test mode, or Paymob sandbox, behind a `PaymentGateway` interface
Tests	Pest
API docs	knuckleswtf/scribe
Local env	Laravel Sail (Docker)
Mail (local)	Mailpit (included in Sail)

In scope: auth, roles, events and ticket types, seat holds, checkout, payment webhook, PDF tickets by email, check-in by QR, tests, CI, docs.

Out of scope: frontend UI, refunds, seat maps (assigned seat numbers). Mention these as future work in the README.

Repo setup

You'll develop natively on Windows with Laravel Herd. The free version gives you PHP, Composer, the Laravel installer, Node and `.test` domains. MySQL, Redis and mail capture are Herd Pro features, so you'll add them with free tools instead.

One-time Windows setup:

1. Install Herd for Windows and choose PHP 8.3 or newer in Herd → Settings → PHP.
2. Install DBngin (free) and create two services: MySQL 8 on port 3306 and Redis on port 6379. Start both. The default MySQL user is `root` with no password.
3. Download Mailpit for Windows, unzip it, and run `mailpit.exe`. The inbox opens at http://localhost:8025.
4. Install Git for Windows. It also gives you Git Bash, which you'll use for the concurrency test.
5. Optional: install a database GUI such as HeidiSQL or TablePlus to browse your tables.
6. Optional, for payments week: install the Stripe CLI to forward webhooks to your local site.

In PowerShell, configure Git and create an SSH key for GitHub:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global core.autocrlf true
ssh-keygen -t ed25519
# then add the contents of $HOME\.ssh\id_ed25519.pub in GitHub → Settings →
SSH keys
```

Create the project. Herd serves every folder in `$HOME\Herd` automatically, so the site will be at http://event-ticketing-api.test.

```
cd $HOME\Herd
laravel new event-ticketing-api
# choose: No starter kit, Pest for testing, MySQL as the database
cd event-ticketing-api
php artisan install:api
```

Install packages:

```
composer require spatie/laravel-permission barryvdh/laravel-dompdf
simplesoftwareio/simple-qrcode stripe/stripe-php
composer require --dev knuckleswtf/scribe larastan/larastan

php artisan vendor:publish --
provider="Spatie\Permission\PermissionServiceProvider"
php artisan vendor:publish --tag=scribe-config
```

Horizon needs the `pcntl` PHP extension, which doesn't exist on Windows. Locally you'll run `php artisan queue:work` instead. If you want Horizon for production, install it with `composer require laravel/horizon --ignore-platform-req=ext-pcntl --ignore-platform-req=ext-posix` and only run it on the Linux server.

Set these values in `.env`:

```
APP_URL=http://event-ticketing-api.test
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=event_ticketing
DB_USERNAME=root
DB_PASSWORD=
REDIS_CLIENT=phpredis
REDIS_HOST=127.0.0.1
QUEUE_CONNECTION=redis
CACHE_STORE=redis
MAIL_MAILER=smtp
MAIL_HOST=127.0.0.1
MAIL_PORT=1025
SEAT_HOLD_MINUTES=10
STRIPE_SECRET=sk_test_xxx
STRIPE_WEBHOOK_SECRET=whsec_xxx
```

Create the `event_ticketing` database in DBngin (or your GUI), then run `php artisan migrate`. Commit `.env.example` with the same keys and empty secrets; never commit `.env`.

Daily workflow. Herd serves the site, so you only need two terminals for background work:

```
php artisan queue:work      # terminal 1: runs queued jobs (PDFs, emails)
php artisan schedule:work   # terminal 2: releases expired seat holds every
minute
```

When testing payments, run `stripe listen --forward-to event-ticketing-api.test/api/v1/webhooks/stripe` in a third terminal and put the `whsec_...` secret it prints into `.env`.

First commit:

```
git init
git add .
git commit -m "chore: bootstrap Laravel 12 with Sanctum, Pest and packages"
git branch -M main
git remote add origin git@github.com:<your-username>/event-ticketing-api.git
git push -u origin main
```

Folder structure

Keep controllers thin: they validate (Form Request), call one Action, and return an API Resource. Business logic lives in Actions, like NestJS services.

```
app/
├── Actions/
|   ├── Bookings/
|   |   ├── HoldSeatsAction.php
|   |   ├── ConfirmBookingAction.php
|   |   └── ReleaseExpiredHoldsAction.php
|   └── Events/
|       └── PublishEventAction.php
├── Contracts/
|   └── PaymentGateway.php          # interface, like a NestJS provider token
├── Enums/
|   ├── BookingStatus.php           # pending, confirmed, expired, cancelled
|   ├── EventStatus.php             # draft, published, cancelled
|   └── TicketStatus.php            # valid, used
├── Events/                         # domain events
|   └── BookingConfirmed.php
├── Exceptions/
|   └── SeatsUnavailableException.php
├── Http/
```

```
|   ├── Controllers/Api/V1/
|   ├── Requests/                   # Form Requests = DTO + ValidationPipe
|   ├── Resources/                  # API Resources = serializers
|   └── Middleware/
├── Jobs/
|   └── GenerateTicketPdfJob.php
├── Listeners/
|   └── SendTicketsListener.php
├── Mail/
|   └── TicketsMail.php
├── Models/
├── Policies/
|   ├── EventPolicy.php
|   └── BookingPolicy.php
├── Providers/
|   └── AppServiceProvider.php      # bind PaymentGateway -> StripeGateway
└── Services/Payments/
    ├── StripeGateway.php
    └── FakeGateway.php             # used in tests
routes/
├── api.php                         # prefix v1
└── console.php                     # scheduler
tests/
├── Feature/
└── Unit/
```

Conventions to follow:

- Use PHP backed enums for every status and cast them in models.
- Store money as integer cents (`price_cents`), never floats.
- Wrap any write touching seats in `DB::transaction()`.
- Return errors in one JSON shape by customizing exception rendering in `bootstrap/app.php`.
- Run `pint` and `phpstan` (level 6) before every push.

Database schema

Eight tables, with users from Laravel's defaults. The key idea: `ticket_types.quantity_sold` and `quantity_held` are counters you update inside a locked transaction.

Table	Key columns	Notes
users	id, name, email, password	roles via Spatie (`organizer` , `attendee` , `admin`)
venues	id, name, address, city, capacity	
events	id, organizer_id, venue_id, title, slug, description, starts_at, ends_at, status	index on (status, starts_at)
ticket_types	id, event_id, name, price_cents, currency, quantity_total, quantity_sold, quantity_held, max_per_order, sales_start_at, sales_end_at	`quantity_total >= sold + held`
bookings	id (uuid), user_id, event_id, status, total_cents, expires_at, payment_intent_id, idempotency_key, confirmed_at	unique on idempotency_key
booking_items	id, booking_id, ticket_type_id, quantity, unit_price_cents	price copied at booking time
tickets	id (uuid), booking_item_id, code (unique), status, checked_in_at, pdf_path	one row per seat
webhook_events	id, provider, provider_event_id (unique), payload (json), processed_at	stops duplicate webhooks

Create them with:

```
php artisan make:model Venue -mf
php artisan make:model Event -mfp
php artisan make:model TicketType -mf
php artisan make:model Booking -mfp
php artisan make:model BookingItem -m
php artisan make:model Ticket -mf
php artisan make:model WebhookEvent -m
```

Example migration for `ticket_types` :

```
Schema::create('ticket_types', function (Blueprint $table) {
    $table->id();
    $table->foreignId('event_id')->constrained()->cascadeOnDelete();
    $table->string('name');
    $table->unsignedInteger('price_cents');
    $table->char('currency', 3)->default('EGP');
    $table->unsignedInteger('quantity_total');
    $table->unsignedInteger('quantity_sold')->default(0);
    $table->unsignedInteger('quantity_held')->default(0);
    $table->unsignedTinyInteger('max_per_order')->default(10);
    $table->timestamp('sales_start_at')->nullable();
    $table->timestamp('sales_end_at')->nullable();
    $table->timestamps();
});
```

Add a `DatabaseSeeder` that creates roles, 2 organizers, 10 attendees and 5 published events. Reviewers will run `php artisan migrate --seed` first.

API endpoints

All routes live under `/api/v1`. Authenticated routes use `auth:sanctum`; role checks use Policies.

Method	Path	Who	Purpose
POST	/auth/register	public	Register as attendee
POST	/auth/login	public	Get a Sanctum token
POST	/auth/logout	auth	Revoke current token
GET	/me	auth	Current user and roles
GET	/events	public	List published events (filter by city, date range, search; paginated)
GET	/events/{slug}	public	Event details with ticket types and seats left
POST	/events	organizer	Create event (draft)

Method	Path	Who	Purpose
PATCH	/events/{event}	organizer (owner)	Update event
POST	/events/{event}/publish	organizer (owner)	Publish event
POST	/events/{event}/ticket-types	organizer (owner)	Add ticket type
POST	/bookings	attendee	Hold seats, returns booking + payment client secret
GET	/bookings	attendee	My bookings
GET	/bookings/{booking}	owner	Booking details and tickets
DELETE	/bookings/{booking}	owner	Cancel a pending booking, releases held seats
GET	/tickets/{ticket}/download	owner	Download PDF ticket
POST	/checkin	organizer	Scan QR code, mark ticket used
GET	/events/{event}/stats	organizer (owner)	Sold, held and revenue per ticket type
POST	/webhooks/stripe	provider	Payment webhook (no auth, signature check)

Example request for `POST /bookings`:

```
{
  "event_id": 3,
  "items": [
    { "ticket_type_id": 7, "quantity": 2 },
    { "ticket_type_id": 8, "quantity": 1 }
  ]
}
```

Send an `Idempotency-Key` header with it; the same key returns the same booking instead of holding seats twice. Rate-limit this route to 10 requests per minute per user.

Core logic

A booking moves through four states: `pending` (seats held) → `confirmed` (paid) or `expired` (hold timed out) or `cancelled` (user cancelled). Only `pending` can change; the other three are final.

1. Holding seats without overselling

Lock the ticket type rows, check availability, then increment `quantity_held`, all in one transaction. Two parallel requests for the last seat will queue on the lock, and the second one fails cleanly.

```
final class HoldSeatsAction
{
    public function __construct(private PaymentGateway $payments) {}

    public function execute(User $user, Event $event, array $items, string
$idempotencyKey): Booking
    {
        if ($existing = Booking::where('idempotency_key', $idempotencyKey)-
>first()) {
            return $existing;
        }

        return DB::transaction(function () use ($user, $event, $items,
$idempotencyKey) {
            $types = TicketType::whereIn('id', collect($items)-
>pluck('ticket_type_id'))
                ->where('event_id', $event->id)
                ->orderBy('id')            // fixed lock order avoids
deadlocks
                ->lockForUpdate()
                ->get()
                ->keyBy('id');

            $total = 0;
            foreach ($items as $item) {
                $type = $types[$item['ticket_type_id']];
                $available = $type->quantity_total - $type->quantity_sold -
$type->quantity_held;
                if ($item['quantity'] > $available) {
                    throw new SeatsUnavailableException($type);
```

```
                }
                $type->increment('quantity_held', $item['quantity']);
                $total += $type->price_cents * $item['quantity'];
            }

            $booking = Booking::create([
                'user_id' => $user->id,
                'event_id' => $event->id,
                'status' => BookingStatus::Pending,
                'total_cents' => $total,
                'expires_at' => now()-
>addMinutes(config('booking.hold_minutes')),
                'idempotency_key' => $idempotencyKey,
            ]);
            // create booking_items with unit_price_cents copied from $types
...

            $booking->payment_intent_id = $this->payments-
>createIntent($booking);
            $booking->save();

            return $booking;
        });
    }
}
```

Create `config/booking.php` returning `['hold_minutes' => env('SEAT_HOLD_MINUTES', 10)]`, and bind `PaymentGateway` to `StripeGateway` in `AppServiceProvider`.

2. Confirming payment by webhook

1. Verify the signature with `STRIPE_WEBHOOK_SECRET`; reject with 400 if it fails.
2. Insert into `webhook_events` using the provider's event id. If the unique index rejects it, return 200 and stop (already processed).
3. In a transaction, lock the booking. If it is not `pending`, stop.
4. Move quantities from `quantity_held` to `quantity_sold`, set status `confirmed`, and create one `tickets` row per seat with a random unique `code`.
5. Dispatch the `BookingConfirmed` event after commit (`ShouldDispatchAfterCommit`).

Edge case to handle and mention in the README: payment arrives after the hold expired. Re-check availability; if seats are gone, mark the booking for refund instead of confirming.

3. Background jobs

- `SendTicketsListener` (queued) listens to `BookingConfirmed` and dispatches `GenerateTicketPdfJob` per ticket.
- The job renders a Blade view with dompdf, embeds the QR code of `ticket.code`, saves to `storage/app/tickets`, and stores `pdf_path`.
- When all PDFs are ready, send `TicketsMail` with them attached. Use `Bus::batch()` with a `then()` callback for this.
- Set `$tries = 3` and `$backoff = [10, 60, 300]` on jobs.

4. Releasing expired holds

In `routes/console.php`:

```
Schedule::job(new ReleaseExpiredHoldsJob)->everyMinute()-
>withoutOverlapping();
```

The job selects pending bookings past `expires_at`, and for each one, in a transaction with locks, decrements `quantity_held` and sets status `expired`.

5. Check-in

`POST /checkin` takes `{ "code": "..." }`. Lock the ticket; if it is `used`, return 409 with `checked_in_at`; otherwise mark it used. Only the event's organizer may check in its tickets.

6. Caching

Cache `GET /events` and `GET /events/{slug}` for 60 seconds in Redis. Do not cache seats left for long; show it from the database or cache it for 5 seconds at most. Clear an event's cache when it is updated, using a model observer.

Testing plan

Aim for about 40 Pest tests and over 80% coverage of `app/Actions`. Swap the payment gateway for `FakeGateway` in tests with `$this->app->bind(PaymentGateway::class, FakeGateway::class)`.

Must-have feature tests:

- [ ] Register, login and logout return the right status codes and token
- [ ] An attendee cannot create events (403)
- [ ] An organizer cannot edit another organizer's event (403)
- [ ] Draft events don't appear in `GET /events`
- [ ] Booking more than available seats returns 422 and changes nothing
- [ ] Booking more than `max_per_order` returns 422
- [ ] The same `Idempotency-Key` twice creates one booking
- [ ] Expired holds are released by the scheduled job (use `$this->travel(11)->minutes()`)
- [ ] A valid webhook confirms the booking and creates one ticket per seat
- [ ] A webhook with a bad signature returns 400
- [ ] The same webhook twice creates tickets only once
- [ ] `BookingConfirmed` dispatches the PDF jobs (`Bus::fake()`, `Event::fake()`)
- [ ] Tickets mail is sent with attachments (`Mail::fake()`)
- [ ] Checking in a used ticket returns 409

The test that sells the project: no overselling under concurrency. Feature tests share one DB connection, so real parallelism needs a script. Add `tests/Concurrency/oversell.sh` that creates an event with 1 seat, then fires 20 parallel requests:

```
seq 20 | xargs -P 20 -I{} curl -s -o /dev/null -w "%{http_code}\n" \
  -X POST http://event-ticketing-api.test/api/v1/bookings \
  -H "Authorization: Bearer $TOKEN" -H "Idempotency-Key: key-{}" \
  -H "Content-Type: application/json" \
  -d '{"event_id":1,"items":[{"ticket_type_id":1,"quantity":1}]}' | sort |
uniq -c
```

Run it in Git Bash, not PowerShell. Expected output: one `201` and nineteen `422`. Put this result in the README; it is your strongest proof.

Git workflow, CI, README and deployment

Work in small feature branches, open a pull request into `main` for each, and use Conventional Commits (`feat:`, `fix:`, `test:`, `docs:`, `chore:`). A clean history is part of what reviewers judge.

Build plan by branch

Week	Branch	Done when
1	`chore/setup`	Sail runs, packages installed, CI green
1	`feat/auth-roles`	Register, login, roles seeded, tests pass
1	`feat/events`	Event and ticket type CRUD with policies, filters, pagination
2	`feat/booking-holds`	Seat holds with locking and idempotency
2	`feat/payments-webhook`	Gateway interface, Stripe test mode, webhook confirms bookings
2	`feat/expire-holds`	Scheduled release of expired holds
3	`feat/tickets-pdf-mail`	Queued PDFs, QR codes, email with attachments
3	`feat/checkin-stats`	Check-in endpoint and organizer stats
3	`test/concurrency`	Oversell script and result in README
3	`docs/readme-api`	Scribe docs, README, deployment

GitHub Actions

Add `.github/workflows/ci.yml`:

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      mysql:
        image: mysql:8.0
```

```
        env:
          MYSQL_DATABASE: testing
          MYSQL_ROOT_PASSWORD: password
        ports: ['3306:3306']
        options: --health-cmd="mysqladmin ping" --health-interval=10s --
health-retries=5
      redis:
        image: redis:7
        ports: ['6379:6379']
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with:
          php-version: '8.3'
          coverage: pcov
      - run: composer install --no-interaction --prefer-dist
      - run: cp .env.example .env && php artisan key:generate
      - run: vendor/bin/pint --test
      - run: vendor/bin/phpstan analyse --memory-limit=1G
      - run: php artisan test --coverage --min=80
        env:
          DB_CONNECTION: mysql
          DB_HOST: 127.0.0.1
          DB_DATABASE: testing
          DB_USERNAME: root
          DB_PASSWORD: password
          REDIS_HOST: 127.0.0.1
```

README outline

1. One-line pitch and a CI badge
2. Features list
3. Architecture: a short diagram of request → Action → transaction → event → queued jobs
4. Engineering decisions: row locking, idempotency keys, webhook dedup, money as cents, holds with expiry
5. Concurrency proof: the oversell script and its output
6. Running locally: `start DBngin and Mailpit, php artisan migrate --seed, php artisan queue:work`

7. API docs link (Scribe, generated with `php artisan scribe:generate`) and a Postman collection
8. Running tests
9. Coming from NestJS: a short table comparing the concepts
10. Future work: refunds, seat maps, waitlist

Deployment

Deploy to Railway or Render (free or cheap tiers support PHP, MySQL and Redis) or a small VPS with Laravel Forge. You need three processes: the web server, `php artisan horizon` for queues, and `php artisan schedule:work` (or a cron running `schedule:run` every minute). Put the live API docs URL at the top of the README.

Before you share the repo

- [ ] `.env` is not committed and no secrets are in history
- [ ] `migrate --seed` works on a fresh clone
- [ ] CI is green and the badge shows it
- [ ] README has the concurrency result and a demo login
- [ ] Pin the repo on your GitHub profile and add it to your CV with one line: "Laravel ticket booking API with row-level locking, idempotent payment webhooks and queued PDF delivery"